

Milestone A, Face Frenzy System

1. Identified Subsystems The Face Frenzy system is composed of several subsystems:

1. **Camera Subsystem:** Captures player images automatically after countdown.
2. **Face Detection Subsystem:** Detects and counts faces using an AI model (quantized YOLO)
 - https://github.com/Xilinx/PYNQ/blob/master/boards/Pynq-Z1/base/notebooks/video/opencv_face_detect_hdmi.ipynb
3. **User Interface:** Displays instructions and feedback on screen.
4. **Input/Output:** Handles buttons and LED signals.
 - OpenCV for Camera Input (into Camera Subsystem)
 - GPIO for LEDs output
 - GPIO Button for Pause/Unpause
 - HDMI for output
5. **Game State Manager:** Controls score, rounds, and game flow.
 - Saves results into some form of a db
6. **Monitoring (Optional):** Allows remote observation of system status (logs, FPS, detections) over the network
 - Local HTTP Server
 - Will read from the db?

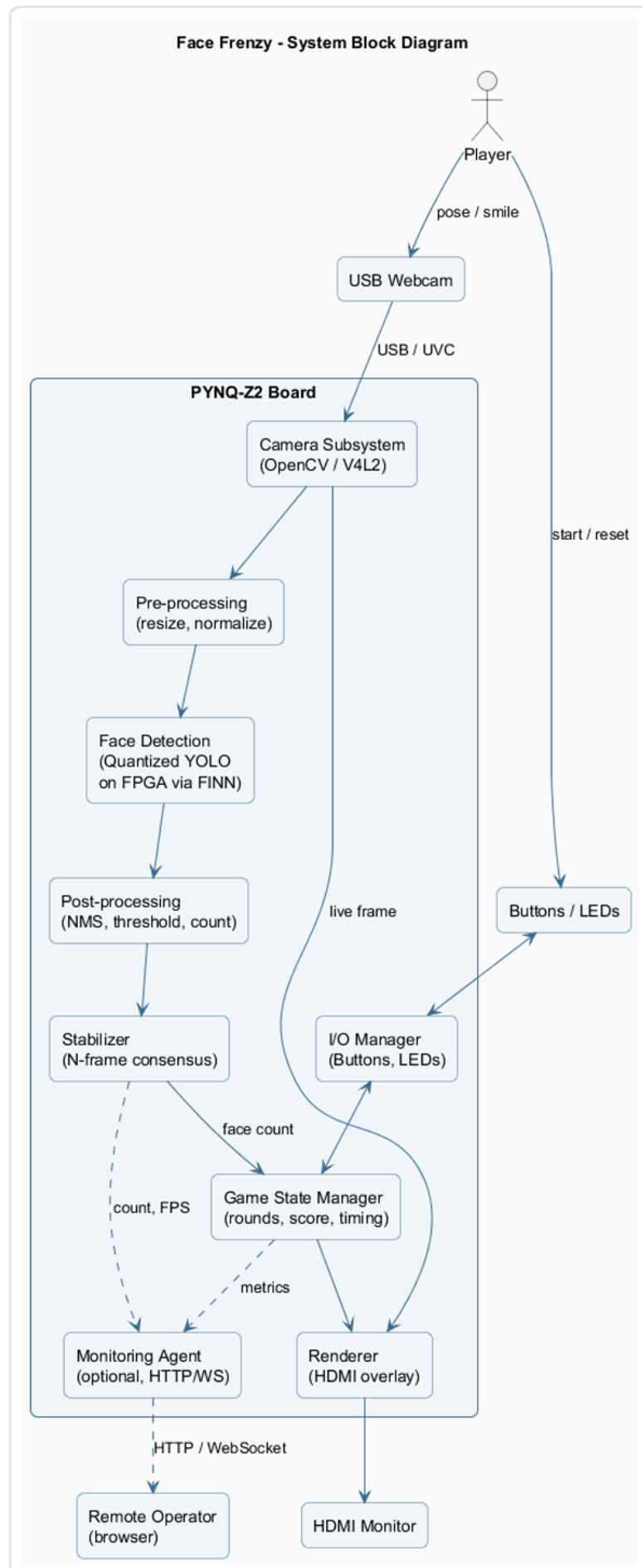


Figure 1 — High-level system block diagram.

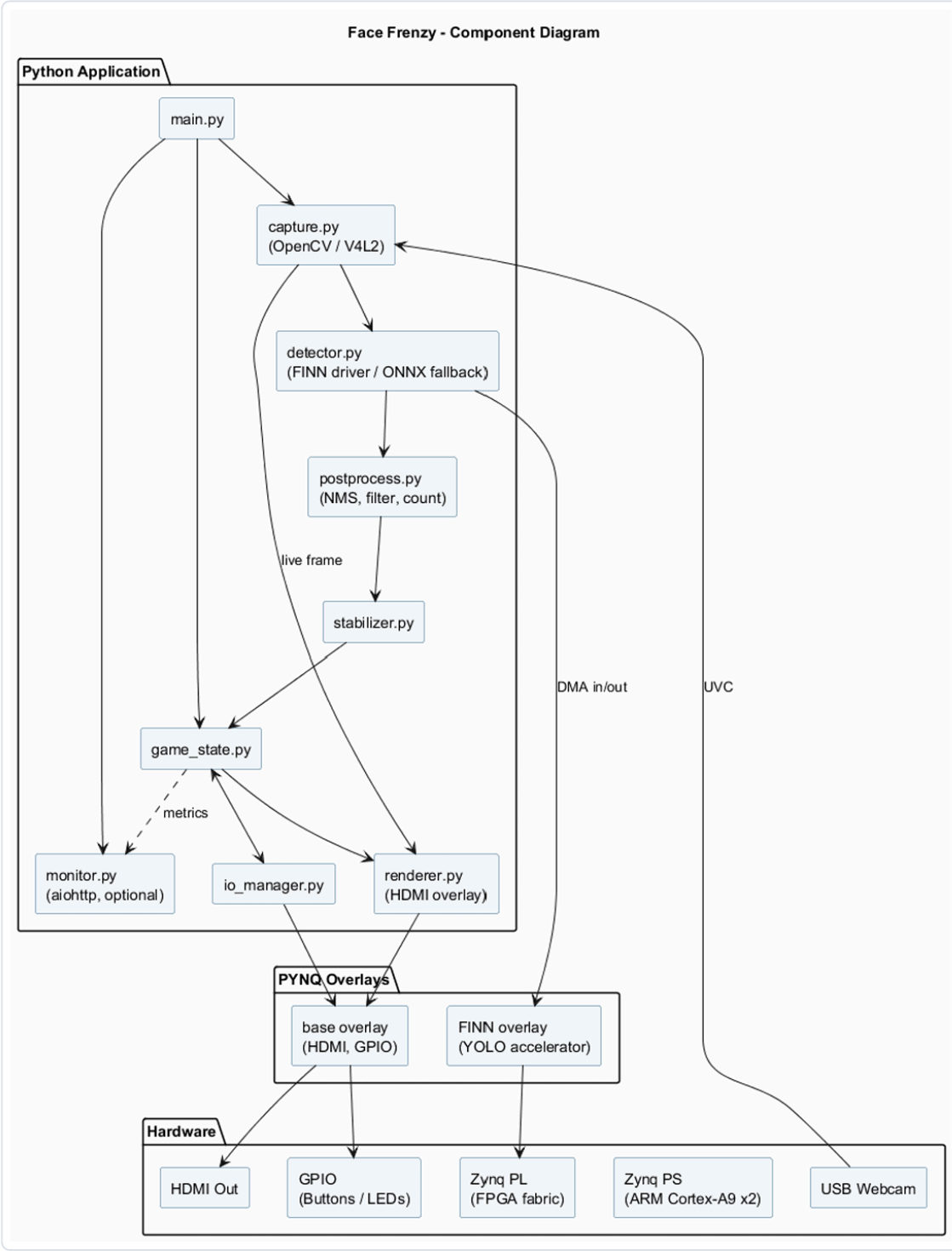


Figure 2 — Hardware / software component view.

2. Feasibility Study Findings

Aspect	Finding
Technical	Achievable using a PYNQ-Z2 board (Zynq-7000), Python (PYNQ overlay APIs, OpenCV) and a pre-trained quantized YOLO model targeting the FPGA via FINN.
Operational	Simple two-button interface and HDMI feedback make it suitable for demos and classroom settings; no training required for end users.
Time	A working prototype (capture → detect → display) is achievable within the milestone window. Tuning latency, accuracy and the optional monitoring service requires additional iterations.
Economic	Uses already-available hardware (PYNQ-Z2, USB webcam, HDMI monitor, LEDs, buttons) and exclusively free / open-source software (Python, OpenCV, FINN, pre-trained model).

3. Key Risks

#	Risk	Likelihood	Impact	Mitigation
R1	Face detection errors due to poor lighting or occlusion	Medium	High	Constrain demo lighting; tune confidence threshold; provide on-screen guidance.
R2	Performance delays in real-time processing	Medium	High	Offload inference to FPGA (FINN); reduce input resolution; pipeline capture/inference/render.

R3	Hardware integration challenges (camera, HDMI, GPIO)	Medium	Medium	Validate each interface independently before integration; rely on existing PYNQ overlays.
R4	Unpredictable user behavior (moving, hiding, extra people)	High	Medium	Add a stabilization window (N consecutive frames must agree) before counting.
R5	Monitoring service degrading game performance	Low	Medium	Run monitoring in a separate process at low priority; throttle update rate.

4. Assumptions

1. Good and reasonably stable lighting conditions during gameplay.
2. Limited number of faces in frame (2–5).
3. Stable hardware setup (camera fixed, monitor connected, no hot-plugging).
4. A pre-trained quantized face-detection model is available, [sefaburakokcu/finn-quantized-yolo](https://github.com/sefaburakokcu/finn-quantized-yolo).
5. Players are roughly facing the camera at a distance of ~1–2 m.

5. Conclusion

The system is feasible using currently available hardware and open-source tooling. The main residual risks concern detection accuracy under non-ideal lighting and end-to-end latency; both are addressable through model tuning and FPGA acceleration. Milestone A confirms the architecture, role assignments and the resource inventory needed to proceed to detailed design and implementation.

Feasibility study—key questions:

Technical feasibility

1. Can the PYNQ-Z2 process video frames in real time using a webcam input?
 - Yes, it is possible to do so; just ensure that the FPGA is utilized properly. (<https://discuss.pynq.io/t/real-time-video-processing-on-pynq-z2/7025>)
2. Is face detection achievable with available overlays, hardware resources, and Python libraries?
 - It is possible, however, that there could be questions about speed, but it could be combated using an overlay.
(Acceleration: <https://github.com/Pomiculture/PYNQ-overlay>)

Operational Feasibility

1. Can the solution operate under classroom lighting and background conditions?
 - It should be possible with minimal error margins. However, due to the camera having auto lighting, it can be hard to say beforehand.
(Study lighting effect on img: <https://share.google/zydi22PTk13m1h9Yq>)
(Camera:

https://www.hp.com/emea_africa-en/products/accessories/product-details/product-specifications/2101129527)

2. Are user interactions intuitive (e.g., visual feedback via HDMI, LED indicators)?

If made correctly, then it should be perfectly intuitive.

Resource Feasibility

1. What are the constraints on CPU, memory, and programmable logic?

1.1 LINK:

<https://www.amd.com/en/corporate/university-program/aup-boards/pynq-z2.html>

- CPU:
 - Zynq-7000 SoC
(<https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html#product-table>)
- MEM:
 - 512 MB DDR3 (16bit bus @1050 Mbps)
 - 128 Mb Quad-SPI Flash
- 2. Are any pre-trained models or external libraries required?
 - OPENCV comes with most of it out the box, however we can use IPython Image

Questions:

- Define what it means for the system to successfully "count faces in real time."
 1. When the count is processed and shown within a few seconds.
- Break down the system into functional blocks
 1. Camera Subsystem
 2. Face Detection Subsystem

3. User Interface
4. Input/Output
5. Game State Manager
6. Monitoring (Optional)

1. How will the webcam stream be accessed? Which library or overlay will handle this?

- Via OpenCV

2. What resolution/frame rate is feasible for the webcam input?

- 30FPS FullHD'

3. What face detection algorithm/model can be used that fits the PYNQ's performance limits?

- YOLO

4. Will detection run on ARM (CPU), or is FPGA acceleration possible or necessary? - FPGA

- It will be able to run on ARM, however the FPGA will significantly improve the speed.

5. How will the system prioritise detection accuracy vs. speed?

- Speed is more important, but accuracy will be maximised until the speed is reached

6. How will the face count be displayed? HDMI overlay? LEDs?

- HDMI overlay

7. What user interaction is needed?

1. Start click
2. Pause click / Unpause
3. Show face(s)

8. How to handle partial faces, faces in motion, or false positives?

- Multiframe consistency check, noise cleaning

9. What can be done to reduce the impact of the monitoring service on the system?

- Make it minimal, don't make it interact with game